AUDITORÍA CRÍTICA · RED-TEAM & DUE DILIGENCE

¿POR QUÉ ATLAS GYM NO VA A FUNCIONAR HOY?

Análisis Escéptico de Riesgos Transaccionales, Operativos, Técnicos y de Modelo de Negocio

PERSPECTIVA	REPOSITORIOS AUDITADOS	DIAGNÓSTICO REAL	FECHA DE REVISIÓN
Inversor / Dueño / Arquitecto	**atlasgym_front + atlasgym_back**	**Alto Riesgo Operativo**	**Octubre 2026**

Premisa del Dictamen Escéptico: Si nos sentamos del otro lado de la mesa como inversores, dueños de un gimnasio con 500 socios o arquitectos senior y se plantea salir a producción en 2 o 3 semanas, la respuesta directa es que **el sistema se va a chocar contra la pared apenas pise un gimnasio real**. No porque la interfaz no sea atractiva o porque falten tests unitarios, sino porque existen bombas de tiempo en la arquitectura de sincronización, en la operatoria diaria de mostrador y en el modelo comercial.

1. La arquitectura de "Doble Verdad" va a hacer desaparecer plata el primer día

RIESGO CRÍTICO · FINANZAS

El frontend nació como un prototipo apoyado en `localStorage` con datos ficticios (`ALUMNOS_MOCK` , `PAGOS_MOCK`) al que luego se le acopló el backend mediante actualizaciones optimistas que **silencian los fallos de red y de servidor**.

- **Evidencia en código:** En `lib/store.tsx` , operaciones críticas como `agregarPago` , `agregarAlumno` , `eliminarAlumno` y `actualizarUsuarioActual` mutan el estado local inmediatamente y encadenan la petición HTTP con `.catch(() ⇒ {})` .
- **Escenario real en mostrador:** Son las 19:30 hs (hora pico). Recepción cobra $35.000 en efectivo; ocurre un microcorte de internet de 2 segundos, expira el token JWT o MySQL devuelve conflicto. La pantalla confirma que **el pago se registró con éxito** porque quedó guardado en el `localStorage` de esa PC y descartó la excepción, pero en MySQL ese cobro **jamás existió**.
- **Consecuencia letal:** A la noche el dueño revisa la caja desde su celular y falta dinero. Al día siguiente, el socio que pagó figura moroso. La primera vez que un sistema contable pierde un cobro o muestra saldos distintos según la pantalla, el dueño vuelve a Excel al día siguiente y abandona el producto.

2. Un único rol ADMIN: invitación abierta al robo hormiga en recepción

RIESGO CRÍTICO · CONTROL INTERNO

En un gimnasio real el dueño no atiende el mostrador las 16 horas de apertura: delega en recepcionistas por turno y profesores de sala.

- **Evidencia en código:** En `Rol.java` solo existen dos roles: `ADMIN` y `ALUMNO`. No existen perfiles intermedios (`RECEPCIONISTA`, `PROFESOR`, `CAJERO`).
- **Vulnerabilidad operativa:** Para que un profesor asigne una rutina o un recepcionista cobre una cuota, hay que otorgarle rol `ADMIN`. Eso le habilita automáticamente `DELETE /api/pagos/{id}` en `PagoController.java`, ver la facturación total del negocio y dar de baja socios.
- **Cero trazabilidad:** Un empleado puede cobrar en efectivo, esperar a que el socio salga por la puerta, eliminar el pago del sistema y quedarse con el billete. No existe ninguna tabla de auditoría (*audit log* inmutable) que registre quién creó, modificó o anuló un movimiento de caja. Ningún dueño serio acepta operar a ciegas con su caja chica.

3. El Tótem no frena morosos, se desincroniza y se auto-bloquea en hora pico

RIESGO CRÍTICO · OPERACIÓN EN PUERTA

El Kiosco Tótem luce excelente en una demostración controlada, pero en la puerta real de un gimnasio falla por cuatro frentes simultáneos:

1. **Los morosos entran igual y no se controlan los días contratados:** En `AsistenciaService.checkInPorDni`, aunque el socio sea `MOROSO`, el backend persiste la asistencia y devuelve `ok: true`. La tablet muestra una alerta amarilla durante 3,5 segundos y se reinicia sola. Sin integración a un molinete físico por relé, en hora pico el moroso pasa caminando. Además, se venden planes de "*3 días por semana*", pero el Tótem jamás contabiliza ingresos semanales: el socio paga 3 días y entrena los 7.
2. **Desincronización entre recepción y entrada:** En `app/totem/page.tsx`, el Tótem evalúa primero la memoria local de la tablet antes de consultar al backend, mientras que `lib/store.tsx` sincroniza pestañas con `BroadcastChannel` (que solo opera dentro del mismo navegador físico). Si el socio paga en la PC del mostrador y camina dos metros hacia la tablet de entrada, la tablet evalúa datos viejos.
3. **Auto-bloqueo (DoS propio) a las 19:00 hs:** En `AsistenciaController.java` se fijó un límite de **20 consultas por minuto por IP**. La tablet del Tótem tiene una única IP (y detrás de Nginx/Docker sin configuración de cabeceras `X-Forwarded-For`, todo el tráfico comparte la IP del proxy). Cuando a las 19:00 hs ingresan 25 socios seguidos (uno cada 2,5 segundos, sumado a errores de tipeo), **al socio N° 21 el propio sistema le bloquea la puerta con HTTP 429**.
4. **Cuentas regaladas por defecto:** En Argentina el DNI es un dato público fácil de obtener. En `AlumnoService.crear`, el usuario se genera como `nombre + últimos 3 dígitos del DNI` y **la contraseña inicial es el propio DNI** sin exigir cambio obligatorio en el primer inicio de sesión. Cualquiera que conozca el DNI de un socio accede a su perfil, teléfono, pagos y rutina.

4. Colapso de performance al escalar (El muro de los 800 socios)

RIESGO ALTO · ESCALABILIDAD

El sistema vuela con una semilla de prueba reducida, pero su diseño de consultas colapsa tras 12 meses de operación real:

- **Problema N+1 masivo en Base de Datos:** `AlumnoService.listarTodos()` itera sobre cada socio y ejecuta `calcularEstadoCuenta(alumno)`, disparando `pagoRepository.findByAlumnoIdOrderByFechaDesc( ... )` individualmente por cada registro. En un gimnasio con 1.000 socios históricos (activos e inactivos), abrir el listado dispara **más de 1.000 consultas SQL separadas** por cada refresco de pantalla.
- **Ausencia absoluta de paginación y límite de 5 MB en navegador:** `PagoService.listarTodos()` y `AsistenciaService.listarTodas()` ejecutan `findAll()` trayendo toda la historia a memoria RAM, y el frontend intenta serializar esas colecciones completas dentro de `localStorage`. Con 150 ingresos diarios (~45.000 asistencias al año), el payload JSON degrada la red y detona `QuotaExceededError` al superar el límite de 5 MB del navegador.

5. El motor comercial no entiende cómo cobra un gimnasio real

RIESGO ALTO · LÓGICA DE NEGOCIO

La función `calcularEstadoCuenta` en `AlumnoService.java` modela una economía idealizada que choca con las promociones y cobros habituales del rubro:

- **Mezcla contradictoria de mes calendario y días corridos:** Verifica simultáneamente `periodoActual.equals(ultimoPago.getPeriodoMes())` (mes calendario `yyyy-MM`) o que la fecha de pago esté dentro de los últimos 31 días. Un socio que paga octubre atrasado el día 28 figura `AL_DIA` hasta el 28 de noviembre sin haber pagado noviembre.
- **Castigo a los mejores clientes (Planes Trimestrales / Semestrales / Anuales):** La entidad `Pago` solo registra un único string `periodoMes` (ej. `"2026-10"`) sin fecha de vigencia (`fechaDesde` / `fechaVencimiento`). Si un socio paga un pack trimestral o anual por adelantado, **al día 32 el sistema lo clasifica automáticamente como MOROSO**.
- **Casos diarios no soportados:** No admite pagos parciales o señas, congelamiento de días por viaje o lesión, recargos automáticos fuera de término ni conciliación por webhook con MercadoPago (obliga a cargar transferencias manualmente).

6. El problema de mercado: Unit Economics y fricción en el piso de entrenamiento

RIESGO ALTO · VIABILIDAD COMERCIAL

- **Arquitectura Single-Tenant inviable como negocio SaaS:** El planteo actual requiere desplegar y mantener una base de datos y contenedores dedicados por gimnasio. En Latinoamérica, un gimnasio independiente paga entre USD 20 y USD 40 mensuales por plataformas *multi-tenant* consolidadas (*Crossfy, MisActividades, Fitco*) que ya incluyen app nativa en tiendas, débito automático, control de molinetes y facturación fiscal (AFIP/ARCA). Administrar servidores aislados por cliente destruye el margen operativo.
- **Fricción real del socio entrenando:** Los gimnasios suelen operar en locales profundos o galpones con mala cobertura 4G. El socio tiene las manos transpiradas, escucha música y necesita registrar cargas en segundos. Si la web-app no cuenta con un *Service Worker* y cola de sincronización offline real, a la tercera serie que pierda por falta de señal cerrará el navegador y volverá a WhatsApp o notas locales.

Matriz Ejecutiva de Puntos de Quiebre

Vector de Falla	Ubicación en Código	Efecto en Operación Real	Impacto
Escritura Optimista Silenciada	`lib/store.tsx` (`.catch(() ⇒ {})`)	Pagos cobrados en mostrador que figuran en la PC local pero desaparecen de MySQL.	**Pérdida de Caja**
Rol Único de Gestión	`Rol.java` / `PagoController.java`	Recepcionistas y profesores pueden borrar pagos en efectivo sin dejar rastro.	**Fraude Interno**
Rate Limit & Caché en Tótem	`AsistenciaController.java` / `totem/page.tsx`	Bloqueo HTTP 429 al socio N° 21 en hora pico; morosos ingresan sin freno real.	**Caída en Puerta**
Consultas N+1 y Sin Paginación	`AlumnoService` / `AsistenciaService`	+1.000 queries SQL por carga y desborde de 5 MB en `localStorage` al año.	**Degradación**
Vigencia Mensual Rígida	`Pago.java` (`periodoMes` único)	Socios con plan trimestral o anual pasan a estado `MOROSO` al día 32.	**Rechazo Comercial**

Veredicto Final: Hoy ATLAS Gym es un **excelente MVP visual y académico**, pero todavía razona como una maqueta de frontend conectada a una base de datos, no como un sistema transaccional diseñado para resistir empleados desleales, microcortes de red, abonos trimestrales y 200 personas ingresando transpiradas a las 19:00 hs. Convertir esta lista crítica en la hoja de ruta de ingeniería es exactamente lo que separa una demo universitaria de un producto comercial vendible.